Module 3 | Numerical Quadrature

The past two chapters have developed a suite of tools for polynomial interpolation and approximation. Now we are going to apply these tools toward the approximation of definite integrals.

Most definite integrals are difficult or impossible to evaluate exactly. For example, one of the important integrals in probability and statistics is the *error function*

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\, dt.$$

which has no closed-form formula for the indefinite integral. Then how does a computer evaluate values for this function?

Integrals also appear intrinsically in approximation theory. For instance, in least squares approximation, we repeatedly encounter inner products of the form

$$\langle f, \varphi_j \rangle = \int_a^b f(x)\varphi_j(x)\, w(x)\, \mathrm{d}x,$$

so even the task of computing a best approximation can reduce to evaluating definite integrals.

So this chapter will be about developing algorithms that approximate such integrals quickly and accurately. The difficulties associated with symbolic integration are so serious that you have already seen some numerical methods for approximating definite integrals in calculus (Riemann Sums). We'll recap some of these in a bit more detail than you might recall, and introduce a few new ideas along the way.

§3.1 Quadrature rules

Since we want to sound more fancy than a calculus course, we will use the exotic new term **quadrature** to refer to numerically approximating definite integrals. The term "quadrature" in math historically refers to finding the area of a geometric figure in the plane by dividing it into shapes of known area (e.g., squares, rectangles or triangles), and many of the numerical integration methods from calculus do just that.

A large class of quadrature rules is built from interpolation/approximation: choose a simple approximant p to f (typically a piecewise polynomial), and then set

$$\int_a^b f(x)\,\mathrm{d}x \approx \int_a^b p(x)\,\mathrm{d}x,$$

reducing quadrature to (i) constructing p from point samples and (ii) integrating p exactly.

3.1.1 Riemann sums

The oldest and simplest examples come from Riemann sums. Fix a partition

$$a = x_0 < x_1 < \cdots < x_n = b, \qquad h_i := x_{i+1} - x_i.$$

Approximating f on each interval $[x_i, x_{i+1}]$ by a constant yields the rectangle rules:

$$\mathrm{Q_L}(f) = \sum_{i=0}^{n-1} h_i f(x_i), \qquad \mathrm{Q_R}(f) = \sum_{i=0}^{n-1} h_i f(x_{i+1}), \qquad \mathrm{Q_M}(f) = \sum_{i=0}^{n-1} h_i f\left(\frac{x_i + x_{i+1}}{2}\right).$$

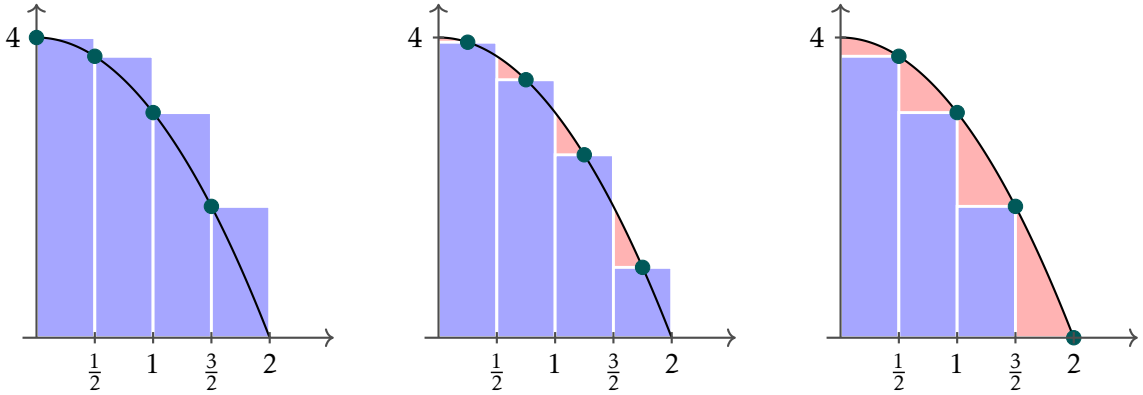


Figure 3.1: Left Riemann sum, midpoint Riemann sum, and right Riemann sum.

While writing the formula for the Riemann sums, you might have observed that each of the methods approximates the definite integral by a weighted average of the function values at an ordered set of points $\{x_0, x_1, \ldots, x_n\}$:

$$\int_a^b f(x)\,\mathrm{d}x \approx \sum_{i=0}^{n} w_i f(x_i)$$

where the coefficients w_i are the weights. Often (but not always), quadrature rules use only points x_i in the interval $[a, b]$. All quadrature methods are essentially about choosing the x_i and w_i in an efficient way.

■ Question 36. □

Suppose we wish to approximate $\mathrm{I}(f) := \int_a^b f(x)\,\mathrm{d}x$ *using a uniform partition* $x_i = a + ih$ *with* $h = (b-a)/n$.

(a) *If* $f \in \mathrm{C}^1([a,b])$, *then prove that*

$$\left|\mathrm{I}(f) - \mathrm{Q_L}(f)\right| \le \frac{b-a}{2}\|f'\|_\infty h, \qquad \left|\mathrm{I}(f) - \mathrm{Q_R}(f)\right| \le \frac{b-a}{2}\|f'\|_\infty h.$$

(b) *If* $f \in \mathrm{C}^2([a,b])$, *then prove that*

$$\left|\mathrm{I}(f) - \mathrm{Q_M}(f)\right| \le \frac{b-a}{24}\|f''\|_\infty h^2.$$

3.1.2 Trapezoidal Rule

You may have heard of the trapezoidal rule for approximating a definite integral.

$$Q_T(f) := \sum_{i=0}^{n-1} \frac{h_i}{2}\Big(f(x_i) + f(x_{i+1})\Big).$$

In the uniform-mesh case $h_i \equiv h$, this becomes

$$Q_T(f) = \frac{h}{2}\Big(f(x_0) + 2\sum_{i=1}^{n-1} f(x_i) + f(x_n)\Big).$$

Here is a picture to describe the process:

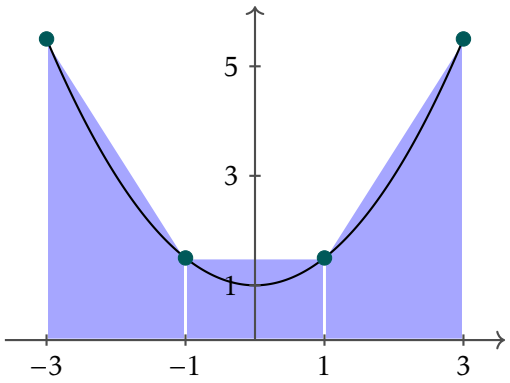


Figure 3.2: Trapezoid rule

Our knowledge about polynomial interpolant gives us a new way to think about this rule. Notice that the trapezoid rule essentially integrates the linear f–interpolation of the endpoints over each subinterval (i.e., it integrates the *linear spline*). It may not have occurred to you that the other three rules (left-endpoint, right-endpoint, and midpoint) can be considered as integration of different 0–degree interpolations (i.e., constants) on each subinterval (so they integrate a kind of pseudo-spline where the endpoints don't necessarily connect).

Note: On a uniform mesh the trapezoidal rule is the average of the left and right rectangle rules:

$$Q_T(f) = \frac{1}{2}\Big(Q_L(f) + Q_R(f)\Big).$$

Question 37. □

Let $f \in C^2([a,b])$ and $x_i = a + ih$ with $h = (b-a)/n$. Then prove that

$$\Big|I(f) - Q_T(f)\Big| \le \frac{b-a}{12}\|f''\|_\infty h^2.$$

What happens if we consider an higher degree spline or a different interpolant? As we will see in lab assignment 3.5, this indeed reduces the error – sometimes by an even greater margin than we might expect.

3.1.3 Newton-Cotes Quadrature

Newton-Cotes quadrature is based on choosing **Lagrange interpolation with equally spaced points**. In that sense, the trapezoidal rule is also a Newton-Cotes quadrature rule that uses a two-point interpolant on each subinterval. Here are some other options.

Simpson's Rule

Suppose we wish to use three-point interpolants on each subinterval, i.e., quadratic splines.

We will start simple: consider a subinterval of the form $[x_0, x_2]$ with midpoint $x_1 = \dfrac{x_0 + x_2}{2}$.

Using the Lagrange basis,

$$\mathrm{P}_2(x) = f(x_0)\,\frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)} + f(x_1)\,\frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)} + f(x_2)\,\frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}$$

Let's integrate our interpolant to find an approximation of the integral:

$$\int_{x_0}^{x_2} f(x)\,\mathrm{d}x \approx f(x_0)\int_{x_0}^{x_2}\frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}\,\mathrm{d}x + f(x_1)\int_{x_0}^{x_2}\frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}\,\mathrm{d}x + f(x_2)\int_{x_0}^{x_2}\frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}\,\mathrm{d}x$$

Assume we have a uniform partition where the width of the subinterval is h. In other words, $x_1 - x_0 = x_2 - x_1 = \dfrac{h}{2}$. Then, after some change of variable calculation, we get

$$\int_{x_0}^{x_2} f(x)\,\mathrm{d}x \approx \frac{h}{6}\left(f(x_0) + 4f(x_1) + f(x_2)\right) =: \mathrm{Q}_{\mathrm{SR}} f$$

The last formula is known as **Simpson's Rule** for approximating definite integrals.

> **Note:** In this formula,
>
> $$w_0 = \frac{h}{6}, \qquad w_1 = \frac{4h}{6}, \qquad w_2 = \frac{h}{6}$$

If we want to use Simpson's rule for more finer partitions, we must first make sure that the number of intervals is even, say, 2M with $h = \dfrac{b-a}{2\mathrm{M}}$. Then we can write

$$\begin{aligned}\int_a^b f(x)\,\mathrm{d}x &= \sum_{k=1}^{\mathrm{M}} \int_{x_{2k-2}}^{x_{2k}} f(x)\,\mathrm{d}x \approx \sum_{k=1}^{\mathrm{M}} \frac{h}{3}\left[f(x_{2k-2}) + 4f(x_{2k-1}) + f(x_{2k})\right] \\ &= \frac{h}{3}\left[f_0 + 4f_1 + 2f_2 + 4f_3 + 2f_4 + \cdots + 2f_{2\mathrm{M}-2} + 4f_{2\mathrm{M}-1} + f_{2\mathrm{M}}\right] \\ &= \underbrace{\frac{h}{3}(f_0 + f_{2\mathrm{M}}) + \frac{2h}{3}\sum_{k=1}^{\mathrm{M}-1} f_{2k} + \frac{4h}{3}\sum_{k=1}^{\mathrm{M}} f_{2k-1}}_{\text{Composite Simpson's Rule}}\end{aligned}$$

Because Simpson's rule captures part of the concavity of an underlying function, it is generally a better approximate than the trapezoid rule.

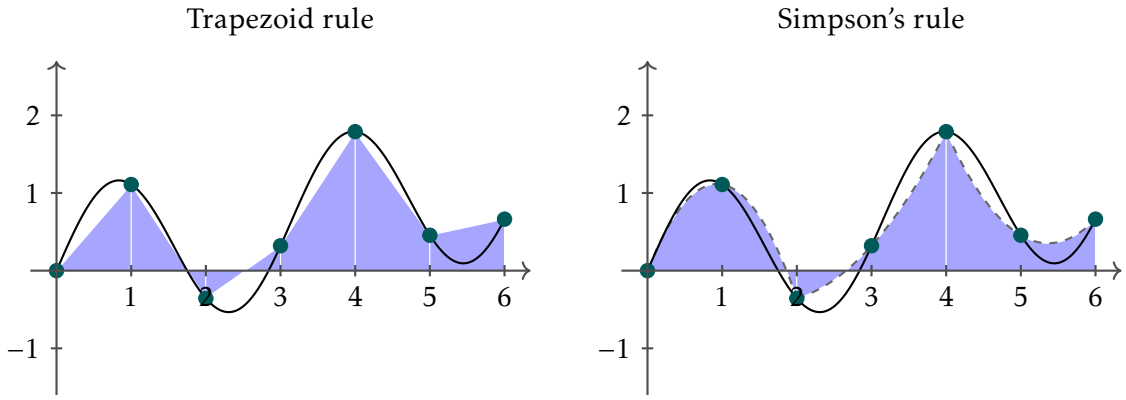


Figure 3.3: Trapezoid vs. Simpson's Rule on $[0,6]$ with 6 equal subintervals for $f(x) = \sin(2x) + 0.2x$.

■ Question 38. □

Write a code to approximate a definite integral using composite Simpson's rule.

Then, use your code to find the smallest number of equally-spaced subintervals that produces a composite Simpson's approximation of

$$\int_0^b x^2 \sin(2x)\,\mathrm{d}x$$

within an error tolerance of* 0.001 *in the following cases:*

(a) *where b is the first zero of the integrand* $x^2\sin(2x)$ *to the right of zero.*

(b) *where b is the second zero of the integrand to the right of zero.*

*Use Mathematica's `NIntegrate` command or Python's quad from scipy.integrate to get the "true" value of the integral.

Using Higher Degree Splines

Using `Mathematica`, the Newton-Cotes quadrature formula for degree 3 and 4 were computed to be as follows:

$\boxed{n=3}$ $$\int_{x_0}^{x_3} f(x)\,\mathrm{d}x \approx \frac{3h}{8}\left[f(x_0)+3f(x_1)+3f(x_2)+f(x_3)\right]$$ ("Simpson's Three-Eights rule")

$\boxed{n=4}$ $$\int_{x_0}^{x_4} f(x)\,\mathrm{d}x \approx \frac{2h}{45}\left[7f(x_0)+32f(x_1)+12f(x_2)+32f(x_3)+7f(x_4)\right]$$

Conclusion. This will increase the number of evaluations of f (often very costly), but hopefully will significantly decrease the error. This is essentially true until $n=8$. After that point, negative weights appear in the Newton-Cotes formulas, and rounding error can become a significant feature of the calculations. For this reason, the Newton-Cotes formulas beyond $n=7$ are rarely used.

Note: Here is another example to emphasize why high-degree Newton–Cotes rules can be a bad idea. Recall that Runge's function $f(x)=\dfrac{1}{1+x^2}$ gave a nice example for which the polynomial interpolant at uniformly spaced points over $[-1,1]$ fails to converge uniformly to f. This fact suggests that Newton–Cotes quadrature will also fail to converge as the degree of the interpolant grows, and indeed it does. On the other hand, integrating the interpolant at Chebyshev nodes, called *Clenshaw–Curtis quadrature*, does indeed converge.

3.1.4 Gaussian Quadrature

In general, we are interested in quadrature rules for *weighted* integrals of the type we saw last chapter,

$$\mathrm{I}_w(f) := \int_a^b f(x)\,w(x)\,\mathrm{d}x \;\approx\; \sum_{i=0}^{n} w_i\,f(x_i),$$

where $w(x)$ is a fixed nonnegative, continuous, integrable weight function on $[a,b]$ (typically $w(x)>0$ a.e.).

The main motivation for weighted quadrature rules is to handle poorly behaved integrands – singular, discontinuous, highly oscillatory, and so on – where the "bad" behavior is known and can be factored out into $w(x)$.

For example, suppose need to estimate integrals of the form

$$\int_{-1}^{1} \frac{f(x)}{\sqrt{1-x^2}}\,\mathrm{d}x, \qquad \int_{-1}^{1} f(x)\,e^{-x}\,\mathrm{d}x,$$

or more generally $\int_a^b f(x)w(x)\,\mathrm{d}x$ where w contains the dominant variation. By designing a quadrature rule with $w(x)$ taken into account, we can obtain fast convergence as long as the remaining factor $f(x)$ is

smooth, regardless of how "bad" $w(x)$ is. Moreover, the rule can be re-used for many different $f(x)$ as long as $w(x)$ remains the same.

Weighted integrals also arise naturally:

- when doing change of variables, since a Jacobian factor becomes a weight; and
- in probability and statistics, since expectations have the form $\mathbb{E}[f(X)] = \int f(x)\mu(x)\,dx$;

This is where **Gaussian quadrature** becomes useful. We will start with a family of orthogonal polynomials $\{\varphi_k\}_{k\geq 0}$ with respect to the inner product with weight function $w \in C[a,b]$.

$$\langle f, g\rangle = \int_a^b f(x)g(x)w(x)\,dx$$

Recall that φ_{n+1} has $(n+1)$ distinct roots x_i. We use these roots as the nodes to construct the Lagrange basis $\{\ell_i\}_{i=0}^n$ and define the quadrature weights by

$$w_i = \int_a^b \ell_i(x)w(x)\,dx$$

Then the resulting rule is

$$\int_a^b f(x)\,w(x)\,dx \approx \sum_{i=0}^n w_i\,f(x_i) = \sum_{i=0}^n \left(\int_a^b \ell_i(x)\,w(x)\,dx\right) f(x_i) =: Q_{G,n}(f),$$

where the nodes x_i are the roots of φ_{n+1}.

Gauss–Legendre Quadrature on General Intervals

When $w(x) = 1$ on $[-1,1]$, the orthogonal polynomials are the Legendre polynomials. The corresponding Gaussian rule is the Gauss–Legendre quadrature rule.

Now, the weights w_i and nodes x_i for Gauss-Legendre quadrature are defined for the reference interval $\hat{I} = [-1,1]$. To evaluate an integral over a general interval $I = [a,b]$, we apply an affine transformation $\varphi : \hat{I} \to I$:

$$\varphi(t) = \frac{b-a}{2}t + \frac{a+b}{2}$$

Substituting $x = \varphi(t)$ into the integral $\int_a^b f(x)\,dx$, we obtain:

$$\int_a^b f(x)\,dx = \frac{b-a}{2}\int_{-1}^1 f\left(\frac{b-a}{2}t + \frac{a+b}{2}\right)dt$$

Consequently, the quadrature rule for $[a,b]$ becomes:

$$Q_{[a,b]}(f) = \frac{b-a}{2} \sum_{i=1}^{n} w_i f(x_i^{rescaled})$$

where $x_i^{rescaled} = \frac{b-a}{2} x_i + \frac{a+b}{2}$.

■ Question 39. □

Using an appropriate weight function $w(x)$ can be useful for integrands with a singularity, since we can incorporate this in $w(x)$ and still approximate the integral with $Q_{G,n}$. Find the Gaussian quadrature $Q_{G,n}$ with $n = 0$, for

$$\int_0^1 \frac{\cos(x)}{\sqrt{x}}\,dx$$

using the weight function $w(x) = \frac{1}{\sqrt{x}}$.

One advantage of this complicated quadrature rule is that the weights are all positive (as opposed to higher order Newton-Cotes). But that's really not enough to justify the complexity of the construction. So, why would anyone do this? To answer that, let's talk about how we measure accuracy of quadratures rules.

§3.2 Degree of Exactness

Observe that if $f \in \mathcal{P}_n$, its polynomial interpolant $p_n \in \mathcal{P}_n$ is exactly f, and so the exact integral of p_n is the same thing as the exact integral of f. However, there are special cases where a degree-n interpolant will exactly integrate polynomials of higher degree. This motivates the next definition.

Definition 3.2.20

Fix a nonnegative, integrable weight function w on $[a,b]$ and define

$$I_w(f) := \int_a^b f(x)\,w(x)\,dx.$$

A quadrature rule

$$Q(f) := \sum_{j=0}^{n} w_j f(x_j)$$

has **degree of exactness** m if

$$Q(p) = I_w(p) \qquad \text{for all } p \in \mathcal{P}_m.$$

Clearly, a degree-n quadrature rule has a degree of exactness at least n. Thus it is particularly interesting to see circumstances in which this degree of exactness is exceeded.

Question 40.

Check that although Simpson's rule is based on quadratic interpolants, it is exact for all cubic polynomials.

The next observation explains why degree of exactness is useful for error estimation. If Q is exact on $\mathcal{P}_k$, then subtracting any $p \in \mathcal{P}_k$ does not change the quadrature error:

$$Q(f) - I_w(f) = Q(f-p) - I_w(f-p).$$

This yields an a priori error bound in terms of best uniform approximation.

Theorem 3.2.21

Suppose w is nonnegative and integrable on $[a,b]$, and suppose the quadrature rule

$$Q(f) = \sum_{j=0}^{n} w_j f(x_j)$$

is exact for all polynomials in $\mathcal{P}_k$. Then for every $f \in C([a,b])$ and every $p \in \mathcal{P}_k$,

$$\left|Q(f) - I_w(f)\right| \le \left(\sum_{j=0}^{n} |w_j| + \int_a^b w(x)\,dx \right) \|f-p\|_{\infty,[a,b]}.$$

Consequently, the problem of error estimates for quadrature is reduced to the previously studied problem of best approximation.

$$\left|Q(f) - I_w(f)\right| \le C \inf_{p\in\mathcal{P}_k} \|f-p\|_{\infty,[a,b]}, \qquad C := \sum_{j=0}^{n} |w_j| + \int_a^b w(x)\,dx.$$

Note: Note that if all quadrature weights w_i are positive, we can choose $C = 2\int_a^b w(x)\,dx$. This implies $Q(f) \to I_w(f)$ as $n \to \infty$. In case of Newton-Cotes, the weights are not positive after $n = 7$, and in fact $\sum w_k \to \infty$. But for Gaussian quadrature, as we will see in a moment, the weights w_k are always positive. So, we have just proved that

$$Q_{G,n} \to I_w(f) \text{ as } n \to \infty$$

Question 41.

Prove that $Q_{G,n}$, the Gaussian Quadrature rule using the roots $\{x_i\}$ of φ_{n+1}, is exact for all polynomials of degree at most $2n+1$, i.e.

$$Q_{G,n}(p) = \int_a^b p(x)\,w(x)\,dx \qquad \text{for all } p \in \mathcal{P}_{2n+1}.$$

Here is a sketch.

- *Given $p \in \mathcal{P}_{2n+1}$, we can find unique $q, r \in \mathcal{P}_n$ such that*

$$p(x) = q(x)\varphi_{n+1}(x) + r(x)$$

- *Show that* $\displaystyle\int_a^b p(x)w(x)\,\mathrm{d}x = \int_a^b r(x)w(x)\,\mathrm{d}x = \underbrace{\sum_{i=0}^{n} w_i r(x_i)}_{Q_{G,n}(r)} = \underbrace{\sum_{i=0}^{n} w_i p(x_i)}_{Q_{G,n}(p)}.$

Question 42.

Prove that the weights in Gaussian quadrature can be also written as

$$w_k = \int_a^b (\ell_k(x))^2\, w(x)$$

This formula is more computationally appealing than the other, because it is more numerically reliable to integrate positive-valued integrands. Also, this clearly shows that $w_k > 0$ for all k.

Our final goal in this section is to provide an error bound formula for $Q_{G,n}(f)$ so that we can analytically argue that a particular numerical estimate is within the degree of accuracy we want. One immediate consequence of the exactness theorem is that we can derive an error formula that depends on $f^{(2n+2)}(\xi)$ for some $\xi \in (a, b)$. To do this, we will need a couple of lemmas.

Theorem 3.2.22: Generalized Mean Value Theorem

Let f and g be continuous real functions on the closed interval $[a, b]$ such that:

$$(\forall x \in [a,b])(g(x) \geq 0) \qquad \text{or} \qquad (\forall x \in [a,b])(g(x) \leq 0)$$

Then there exists a real number $\xi \in [a, b]$ such that:

$$\int_a^b f(x)g(x)\,\mathrm{d}x = f(\xi)\int_a^b g(x)\,\mathrm{d}x$$

You will be using GMVT to answer several questions in the upcoming lab.

We will also make a claim without proof.

Claim 3.2.23: Hermite Interpolation Theorem

Let $n \geq 0$, and suppose that $x_i, i = 0, \ldots, n$, are distinct real numbers. Then, given two sets of real numbers $y_i, i = 0, \ldots, n$, and $z_i, i = 0, \ldots, n$, there is a unique polynomial p_{2n+1} in $\mathcal{P}_{2n+1}$ such that

$$p_{2n+1}(x_i) = y_i, \quad p'_{2n+1}(x_i) = z_i, \quad i = 0, \ldots, n.$$

Note: The above result is yet another way to find a polynomial interpolant that we skipped over in Module 1. You can find the proof in the online textbook I posted to Canvas.

Also, recall how we found the Cauchy Interpolation Error formula from theorem 4. We are going to do the same trick we did for its proof to derive a similar error bound. Define

$$\mathrm{E}_n(x) = f(x) - p_{2n+1}(x)$$

Since both E_n and E_n' are zero at each node, we know that each node has multiplicity 2 as a root. So

$$\mathrm{E}_n(x) = r(x) \cdot \prod_{i=1}^{n} (x - x_i)^2$$

Then using a similar trick as before with a twisted $\hat{\mathrm{E}}_n(x)$, we derive

$$f(x) - p_{2n+1}(x) = \frac{f^{(2n+2)}(\xi)}{(2n+2)!} \prod_{i=1}^{n} (x - x_i)^2,$$

■ **Question 43.** □

Given $f \in \mathrm{C}^{2n+2}[a,b]$, prove that there exists $\xi \in (a,b)$ such that

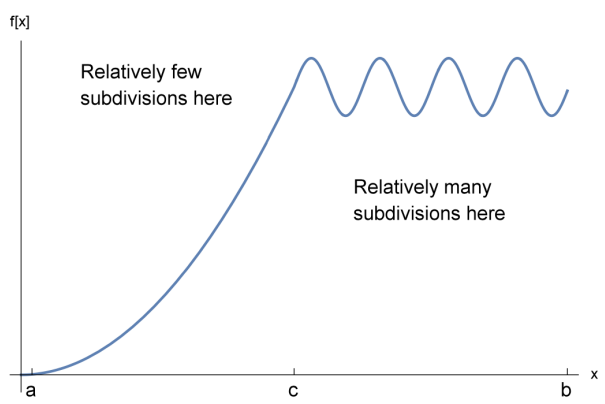
$$\mathrm{I}_w(f) - \mathrm{Q}_{\mathrm{G},n}(f) = \frac{f^{(2n+2)}(\xi)}{(2n+2)!} \int_a^b \varphi_{n+1}^2(x) w(x)\,\mathrm{d}x$$

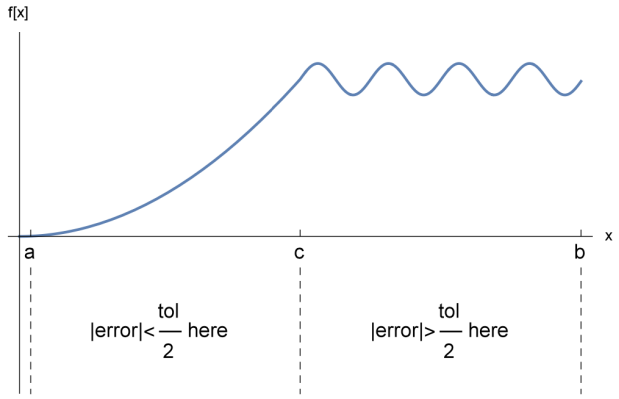
Reminder to myse
talk about feedbac

§3.3 Adaptive Quadrature

We now understand that the approximations given by different quadrature rules can be improved differently by taking smaller subdivisions. However, smaller subdivisions take more computational effort, so in practice, we want to be careful about how we do this. One idea that is used by most commercial numerical integration solvers is called adaptive quadrature, because the size of the subdivisions are adapted as the method progresses.

The basic idea behind adaptive quadrature is to use more subdivisions only on the parts of the interval $[a,b]$ where necessary. For instance, if a function is relatively flat and straight over one section of the interval $[a,b]$, we might not need many subdivisions to get an accurate approximation of the integral. If that same function is very wavy over some other section of the interval $[a,b]$, more subdivisions may be needed there.





In practice, we don't have an image like this to help us to decide where more subdivisions may be needed. Instead, we decide where to do more subdivisions by cleverly comparing local error estimates.

Adaptive Simpson's method. On an interval $[a,b]$ with midpoint $m = \frac{a+b}{2}$, Simpson's rule is

$$\mathrm{S}(a,b) = \frac{b-a}{6}\Big(f(a) + 4f(m) + f(b)\Big).$$

If we split $[a,b]$ into two halves and apply Simpson on each half, we obtain

$$\mathrm{S}(a,m) + \mathrm{S}(m,b).$$

For smooth f, Simpson's error scales as a multiple of $(b-a)^5$, so halving the interval reduces the leading error term by a factor $2^5 = 32$. From this, we can derive the standard error estimator

$$\mathrm{err}(a,b) \;=\; \frac{1}{15}\Big|\Big(\mathrm{S}(a,m) + \mathrm{S}(m,b)\Big) - \mathrm{S}(a,b)\Big|.$$

Algorithm

- **INPUT**: Endpoints a, b, some tolerance tol, a big limit N to number of steps.
- **OUTPUT**: Approximate value of the integral, final number of sampling points used, or message that N was exceeded.
- Let $c = \dfrac{a+b}{2}$. Compute the Simpson approximation on $[a,b]$:

$$\mathrm{S}(a,b) = \frac{b-a}{6}\,[f(a) + 4f(c) + f(b)].$$

- Compute $\mathrm{S}(a,c)$ and $\mathrm{S}(c,b)$ and estimate the local error by

$$\mathtt{error} = \frac{1}{15}\,|(\mathrm{S}(a,c) + \mathrm{S}(c,b)) - \mathrm{S}(a,b)|.$$

- If $\mathtt{error} \le \mathtt{tol}$, stop. Otherwise, recursively apply the same procedure on $[a,c]$ and $[c,b]$ with tolerance $\mathtt{tol}/2$ on each subinterval.

The recursion builds a subdivision of $[a,b]$ that is fine where the integrand is difficult and coarse where it is easy. Notice that Simpson's rule $S(a,b)$ already evaluates $f((a+b)/2)$. So, each refinement step introduces only two new sample points (the quarter-points), and previously computed values are reused. This reuse is essential for efficiency and is part of what makes adaptive quadrature practical.

Digression

There are lots of other quadrature techniques that we do not have time to go over in this course, such as the **Monte Carlo quadrature** (uses a random sampling based statistical approach). In this method, we compute the integral of $f : \mathbb{R}^d \to \mathbb{R}, d \geqslant 1$ by generating n random points in $\mathbb{R}^d$ and use the approximation,

$$\iint \dots \int_{\Omega} f(x_1, x_2, \dots, x_d)\, dx_1\, dx_2 \dots dx_d \approx \text{volume}(\Omega) * \frac{\sum_{i=1}^{n} f(\mathbf{z_i})}{n}$$

where $\mathbf{z_i}$ are randomly chosen values from $\mathbb{R}^d$. Please feel free to choose one as your final project topic.

§3.4 Big O Notation

Definition 3.4.24: Big "$\mathcal{O}$" notation

We write a function $f(h)$ as $\mathcal{O}(h^m)$(for an integer $m \geq 1$) if $f(h)$ can be written as an infinite series of the form

$$f(h) = C_m h^m + C_{m+1} h^{m+1} + \dots \tag{3.1}$$

in terms of some constant coefficients C_m, C_{m+1}, etc.

For our purposes here, the importance of this property is that if an error function $E(h) = \mathcal{O}(h^m)$, then $E(h)$ diminishes like a multiple of h^m as h diminishes. The key principle here is that the lowest power m dominates the infinite series (3.1) when h is small; and small values of h are precisely where we expect our best approximations.

Note: This notation simplifies algebraic manipulations because it represents a category and not a particular element of a category. For instance, we have for any constant multiple $c \neq 0$ that

$$c\mathcal{O}(h^m) = \mathcal{O}(h^m), \quad \mathcal{O}(h^m) + \mathcal{O}(h^m) = \mathcal{O}(h^m), \quad \mathcal{O}((ch)^m) = \mathcal{O}(h^m),$$

because the corresponding infinite series (3.1) all still have lowest exponent m.